

LinguaPlay — Project Report

Overview

LinguaPlay is a language-learning web application that turns any YouTube video into an interactive language class. A user pastes a YouTube link, and the app transcribes the audio, detects the language, analyses the content for linguistic complexity, and opens a conversational interface where the user can ask questions about grammar, vocabulary, idioms, and cultural references — all grounded in what was actually said in the video.

The motivation behind the project was to address a gap in language learning tools: most apps rely on curated, textbook-style content, while real language acquisition happens through authentic material. LinguaPlay lets learners work with content they actually want to watch, whether that's a German science channel, a French cooking show, or a Spanish news segment.

Technical approach

Architecture

The application is split into a Python/FastAPI backend and a React/Vite frontend, communicating over a REST API.

Processing pipeline (triggered when a user submits a URL):

1. Audio extraction — yt-dlp downloads the audio track from the YouTube video
2. Transcription — OpenAI Whisper (whisper-1) transcribes the audio with segment-level timestamps and detects the spoken language
3. Chunking & vector storage — the transcript is split into chunks and stored in Pinecone (serverless, cosine similarity) using OpenAI text-embedding-3-small embeddings
4. Analysis — GPT-4o-mini analyses the full transcript and returns structured output: CEFR difficulty level, notable expressions and idioms, a cultural note, and key vocabulary. Structured output is enforced via Pydantic and LangChain's `with_structured_output()`

Conversational agent:

User messages are handled by a LangGraph agent built with `create_react_agent` and an `InMemorySaver` checkpointer for conversation memory. The agent has access to two tools: `search_transcript` (RAG over Pinecone) and `generate_quiz` (generates an interactive multiple-choice quiz grounded in the transcript). Session isolation is achieved by scoping the LangGraph `thread_id` to a combination of user session ID and video ID.

Frontend:

The React interface is organised into three tabs — Chat, Transcript, and Vocabulary. The chat tab supports both text and voice input (Web Speech API), with bilingual mic buttons automatically set to English and the detected video language. Quiz responses from the agent are rendered as an interactive multiple-choice component rather than plain text.

Key design decisions

Structured output for quiz generation. The quiz tool uses a custom Pydantic schema with separate `correct_answer` and `distractors` fields. Answer options are assembled in Python code (at a random position) rather than relying on the LLM to produce them consistently — this prevents the model from accidentally leaking the correct answer's position and avoids string-matching issues between generated options.

Lazy Pinecone initialisation. The Pinecone client is instantiated on first use rather than at import time, so the server starts cleanly even if environment variables are missing or misconfigured.

RAG grounding. All agent answers are grounded in retrieved transcript chunks rather than the model's general knowledge. This keeps explanations contextually relevant — when a user asks about an expression, the agent explains it in the context of how it was actually used in the video.

Stack

Layer	Technology
Frontend	React, Vite
Backend	FastAPI, Python
AI agent	LangChain, LangGraph
Transcription	OpenAI Whisper (whisper-1)
Embeddings	OpenAI text-embedding-3-small
Vector store	Pinecone (serverless)
LLM	OpenAI GPT-4o-mini
Audio	yt-dlp + FFmpeg
Observability	LangSmith

Challenges

Quiz answer consistency. The first implementation asked the LLM to produce a list of four options including the correct answer. The model would often generate options that were slightly paraphrased from the correct answer, causing string comparison to fail even when the user selected the right one. The solution was to redesign the schema: the model provides a `correct_answer` and three distractors separately, and the options list is assembled in Python.

Cross-lingual retrieval. Transcript chunks are embedded in the video's language (e.g. German). When a user asks a question in English, the English query embedding does not reliably match

German chunk embeddings, so the agent sometimes fails to retrieve relevant context. The current workaround is asking questions in the video's language; a proper fix would involve translating queries before embedding, which is noted as a future improvement.

Deployment bot detection. YouTube actively blocks yt-dlp requests from datacenter IP addresses (Render's servers) and requires authentication cookies. This was resolved by exporting browser cookies and passing them to yt-dlp via an environment variable.

Known limitations

- In-memory session state. Conversation history and processed video data are stored in memory. Restarting the server clears everything — users need to reprocess a video if the server restarts. A production version would persist this to a database.
- Single-user design. There is no authentication layer or multi-user data isolation beyond session IDs. The app is designed for personal use or demonstration.
- Processing time. Transcribing a 10–20 minute video takes 30–90 seconds, with Whisper being the main bottleneck. There is no streaming progress update — the UI shows a loading indicator for the full duration.
- Cross-lingual queries. As noted above, querying in a different language than the video degrades retrieval quality.
- Cookie-based YouTube auth. The deployed version relies on exported browser cookies to bypass YouTube's bot detection. These cookies expire and need to be refreshed periodically.

Potential next steps

- Translate user queries to the video language before embedding, and return answers in the user's preferred language
- Persist session state and processed videos to a database (e.g. PostgreSQL + pgvector) to survive server restarts
- Add automated tests: pytest for the backend pipeline and agent tools, Vitest + React Testing Library for the frontend
- Stream transcription progress to the frontend so users get feedback during the processing wait